

RACIOCÍNIO ALGORÍTMICO

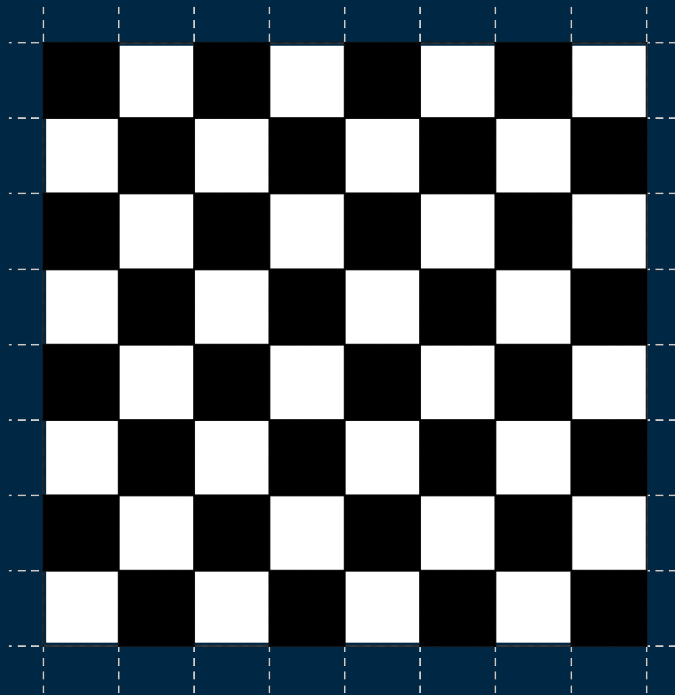
Matrizes

Matrizes

Matrizes são muito semelhantes à listas. Possuem índices para suas “caixinhas”, onde são armazenados os valores.

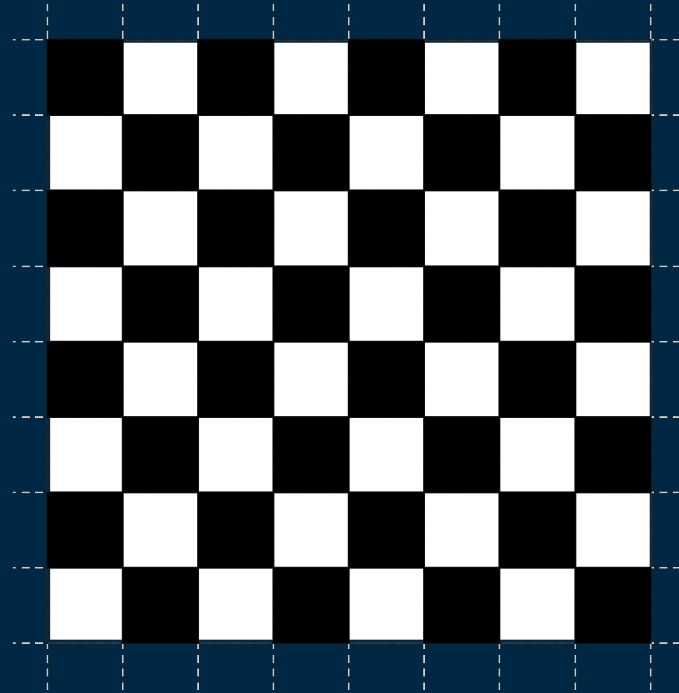
A diferença é que as matrizes possuem dois índices, que são referentes às **linhas** e **colunas**.

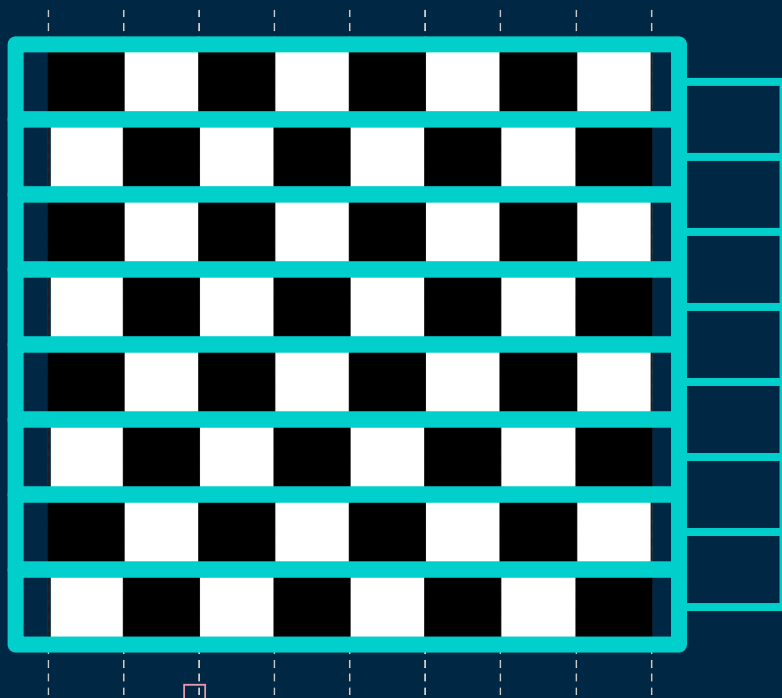
Observe o tabuleiro de Xadrez ao lado:



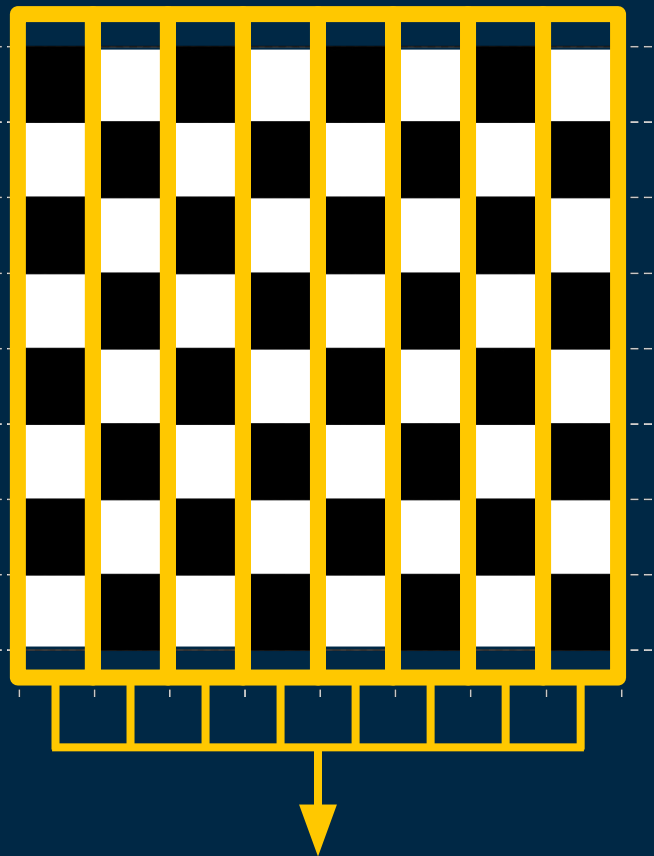
Matrizes

O tabuleiro de xadrez pode ser representado, computacionalmente, por uma matriz de tamanho 8x8, que contém **linhas** e **colunas**.





LINHAS



COLUMNAS

Índices de uma Matriz

Os índices das **linhas** são representados ao lado, verticalmente, no eixo y. E os índices das **colunas** são representados acima, horizontalmente, no eixo x.

	0	1	2	3	4	5	6	7
0								
1								
2								
3								
4								
5								
6								
7								

Índices de uma Matriz

No caso, se quisermos acessar a posição do canto superior direito, poderíamos informar o índice **0** para **linha** e **7** para **coluna**.

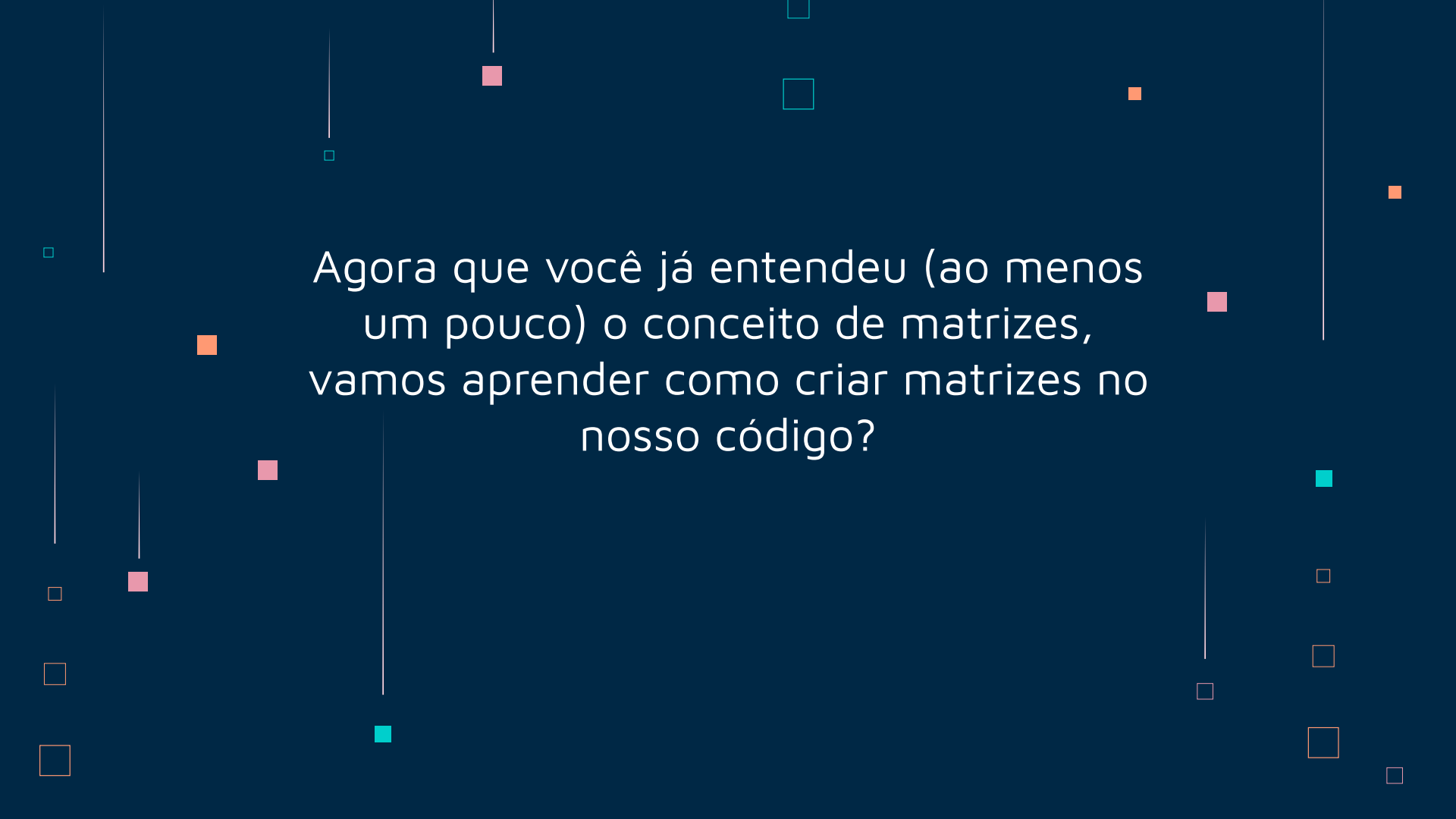
	0	1	2	3	4	5	6	7
0								
1								
2								
3								
4								
5								
6								
7								

Índices de uma Matriz

Vamos acessar algumas outras posições para entender melhor.

- **linha:** 2, **coluna:** 5
- **linha:** 4, **coluna:** 2
- **linha:** 6, **coluna:** 6
- **linha:** 0, **coluna:** 0
- **linha:** 7, **coluna:** 3

	0	1	2	3	4	5	6	7
0								
1								
2								
3								
4								
5								
6								
7								

The background is a dark blue gradient. It features several vertical white lines of varying lengths. Scattered throughout are small squares in teal, orange, and pink. Some squares are solid, while others are outlines. The text is centered in a white, sans-serif font.

Agora que você já entendeu (ao menos um pouco) o conceito de matrizes, vamos aprender como criar matrizes no nosso código?

Criando Matrizes em Python

Uma matriz pode ser pensada como uma lista bidimensional, pois possui duas dimensões (**linhas** e **colunas**)

Observe o exemplo ao lado.

Primeiro, temos a declaração de uma lista simples. Lembra delas?

Criando uma lista:

```
lista = [1, 2, 3, 4]
```

Criando Matrizes em Python

E se ao invés de ter os números um ao lado do outro em uma lista, eu quiser colocar os números 3 e 4 embaixo dos números 1 e 2?

Para isso eu teria que ter uma lista, que possui duas outras listas dentro, ou seja, uma matriz!

Observe no exemplo ao lado.

Criando uma matriz:

```
matriz = [[1, 2], [3, 4]]
```

Criando Matrizes em Python

No exemplo anterior, ainda não conseguimos visualizar os números 3 e 4 embaixo dos números 1 e 2.

Para uma melhor visualização das nossas matrizes no código, podemos representar da seguinte forma, observe o exemplo ao lado.

Criando uma matriz:

```
matriz = [  
    [1, 2],  
    [3, 4]  
]
```

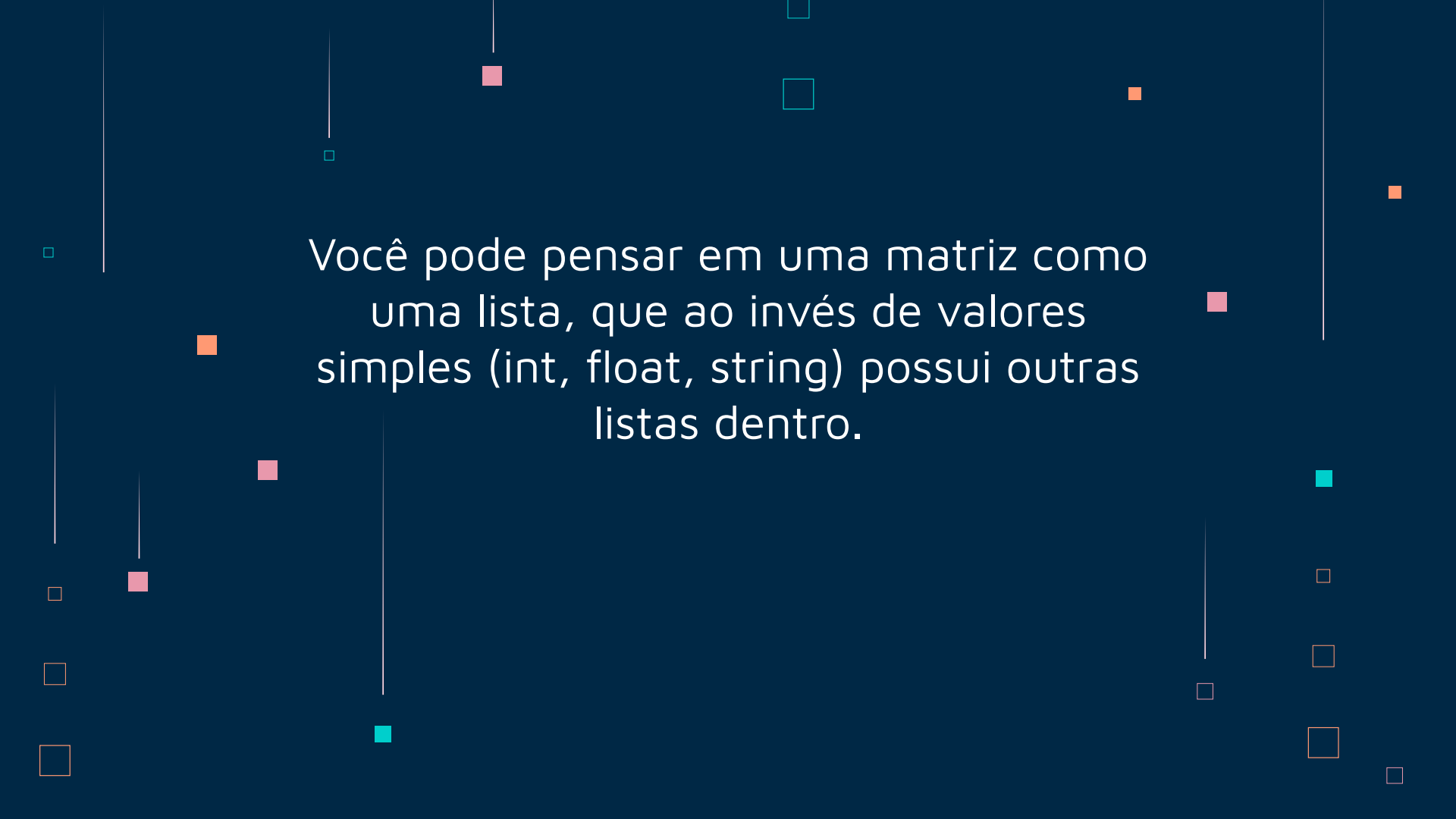
A

```
matriz = [[1, 2], [3, 4]]
```

B

```
matriz = [  
    [1, 2],  
    [3, 4]  
]
```

Os dois exemplos são iguais. Em ambos temos matrizes de tamanho 2x2 que funcionam igualmente no código. A única diferença é que no exemplo **B** obtemos uma melhor visualização das **linhas** e **colunas**.

The background is a dark blue gradient. It features several vertical white lines of varying lengths. Scattered throughout are small squares in light blue, orange, and pink. Some squares are solid, while others are outlines. The text is centered in a white, sans-serif font.

Você pode pensar em uma matriz como
uma lista, que ao invés de valores
simples (int, float, string) possui outras
listas dentro.

Outra forma de visualizar

linha 0

```
matriz = [  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0]  
]
```

coluna 0

```
matriz = [  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0]  
]
```

Outra forma de visualizar

linha 1

```
matriz = [  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0]  
]
```

coluna 1

```
matriz = [  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0]  
]
```


Outra forma de visualizar

linha 2

```
matriz = [  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0]  
]
```

coluna 2

```
matriz = [  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0]  
]
```

Outra forma de visualizar

linha 3

```
matriz = [  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0]  
]
```

coluna 3

```
matriz = [  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0]  
]
```

Alterando valores na Matriz

```
matriz = [  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0]  
]
```

Vamos alterar alguns valores da nossa matriz.

Tente descobrir as posições de **linha** e **coluna** dos valores em **vermelho**, **verde** e **rosa** antes de passar para o próximo slide.

Alterando valores na Matriz

```
matriz = [  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0],  
    [0, 0, 0, 0]  
]
```

- **linha:** 0, **coluna:** 2
- **linha:** 2, **coluna:** 0
- **linha:** 3, **coluna:** 3

Agora vamos alterar (por código) o valor em **vermelho** para **2**, o valor em **verde** para **7** e o valor em **rosa** para **8**.

Alterando valores na Matriz

```
matriz = [  
    [0, 0, 2, 0],  
    [0, 0, 0, 0],  
    [7, 0, 0, 0],  
    [0, 0, 0, 8]  
]
```

- `matriz[0][2] = 2`
- `matriz[2][0] = 7`
- `matriz[3][3] = 8`

Acessamos a posição da matriz com dois colchetes:

- o primeiro para **linha**
- o segundo para **coluna**.

matriz[linha][coluna]

Salvar valores da Matriz em variáveis

```
matriz = [  
    [0, 0, 2, 0],  
    [0, 0, 0, 0],  
    [7, 0, 0, 0],  
    [0, 0, 0, 8]  
]
```

Caso você queira salvar os valores em variáveis é simples, observe:

- `numero1 = matriz[0][2]`
- `numero2 = matriz[2][0]`
- `numero3 = matriz[3][3]`

Percorrendo uma Matriz com *for*

Se para acessar todos os índices de uma lista precisamos de um único *for*, para acessar todos os índices de uma matriz vamos precisar de dois *for*'s

Vamos recapitular o *for* com lista, veja no exemplo ao lado.

Código:

```
lista = [9, 8, 7, 6]
for indice in range(4):
    print(lista[indice])
```

Output:

```
9
8
7
6
```


Percorrendo uma Matriz com *for*

Agora vamos usar **dois** *for* para percorrer uma matriz.

Output:

9
8
7
6

Código:

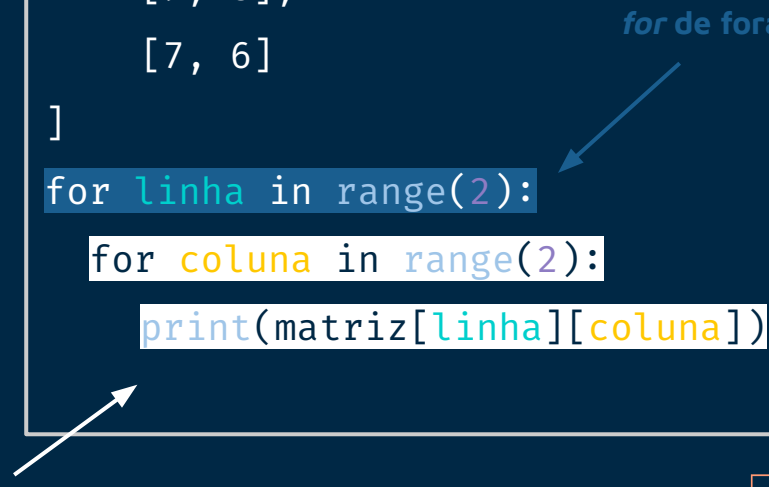
```
matriz = [  
    [9, 8],  
    [7, 6]  
]  
  
for linha in range(2):  
    for coluna in range(2):  
        print(matriz[linha][coluna])
```

Percorrendo uma Matriz com *for*

Lembre que SEMPRE o *for* de dentro (**coluna**) irá rodar em loop ANTES de voltar para o *for* de fora (**linha**).

Código:

```
matriz = [  
    [9, 8],  
    [7, 6]  
]  
for linha in range(2):  
    for coluna in range(2):  
        print(matriz[linha][coluna])
```



```
matriz = [  
    [9, 8],  
    [7, 6]  
]  
  
for linha in range(2):  
    for coluna in range(2):  
        print(matriz[linha][coluna])
```

Teste de mesa:

linha	coluna	matriz[linha][coluna]
0	0	matriz[0][0] = 9
0	1	matriz[0][1] = 8
1	0	matriz[1][0] = 7
1	1	matriz[1][1] = 6

Percorrendo uma Matriz com *for*

Caso você queira imprimir uma lista abaixo da outra, utilize apenas **um** *for* e imprima a lista de cada **linha**.

Código:

```
matriz = [  
    [9, 8],  
    [7, 6]  
]  
  
for linha in range(2):  
    print(matriz[linha])
```

Output:

```
[9, 8]  
[7, 6]
```

Prática 1

Crie uma matriz de tamanho 3x3 e preencha-a com valores de 1 a 9.

Imprima a matriz das três formas:

1. `print(matriz)`
2. utilizando **um** *for*.
3. utilizando **dois** *for*'s.



Prática 2

Utilizando a mesma matriz da prática anterior, altere os valores nas seguintes posições:

1. `[0][0]` para 20
2. `[1][2]` para 15
3. `[2][1]` para 19

Imprima novamente a matriz das 3 formas solicitadas anteriormente.

Obs: Mantenha as 3 impressões da matriz original e faça um novo código para imprimir a matriz com os novos valores que foram alterados.



Prática 3

Faça as seguintes operações aritméticas com os valores de cada posição e armazene-as em variáveis:

1. soma de $[0][0]$ e $[1][0]$
2. subtração de $[2][2]$ e $[2][1]$
3. multiplicação de $[0][1]$ e $[2][0]$
4. divisão de $[1][2]$ e $[0][2]$

Imprima todas as variáveis.

